Plan de Monetización — DominioTotal v1.0

Integración Stripe + Planes Premium

Fecha: 18 de Febrero 2026 | **Estado:** PENDIENTE (Mes 3) | **Meta:** $2,000 MRR en Mes 6

🎯 Resumen Ejecutivo

Objetivo: Lanzar el plan Premium de $40/mes para agentes inmobiliarios en el Mes 3, generando $2,000 MRR para el Mes 6 con 50 agentes de pago.

Stack: Stripe Checkout + Stripe Customer Portal + Webhooks + Firebase
Tiempo de implementación: ~2 semanas
ROI proyectado: $180K ARR al mes 12

💰 Estructura de Precios

Tier 1: Free (Siempre gratis)

Precio: $0/mes
Target: Agentes que recién empiezan, propietarios particulares

Feature	Free
Propiedades activas	3
Fotos por propiedad	5
Leads recibidos	Ilimitados
Dashboard básico	✅
Notificaciones email	✅
Soporte	Community

Tier 2: Premium (Core product)

Precio: $40 USD/mes | $400 USD/año (2 meses gratis)
Target: Agentes activos con 5-20 propiedades

Feature	Premium
Propiedades activas	Ilimitadas
Fotos por propiedad	20
Leads con datos completos	✅
Analytics avanzado	✅
Badge "Agente Verificado"	✅
Destacar propiedades	2/mes
Prioridad en búsqueda	✅
CRM básico (pipeline)	✅
Export leads CSV	✅
Soporte	Email prioritario

Tier 3: Enterprise (Futuro Mes 9)

Precio: $150 USD/mes
Target: Inmobiliarias con 20+ propiedades

Feature	Enterprise
Todo Premium	✅
Propiedades ilimitadas	✅
Destacados ilimitados	✅
API access	✅
Multi-usuario (5 agentes)	✅
White-label light	✅
Soporte	1-on-1 mensual

🔧 Arquitectura Técnica

Stack

```
Stripe Checkout → Webhook → Firebase → Feature Flags → UI
    ↓
Stripe Portal  ← Customer management
```

Diagrama de flujo de suscripción

```
1. Agente hace click en "Upgrade"
2. → Stripe Checkout (hosted page)
3. → Usuario paga
4. → Stripe envía webhook `checkout.session.completed`
5. → Webhook actualiza users/{uid}.subscription en Firestore
6. → Firebase Rules valida el tier en tiempo real
7. → UI muestra features premium automáticamente
```

📁 Implementación Paso a Paso

Paso 1: Setup Stripe (30 min)

```bash
npm install stripe @stripe/stripe-js
```

Variables de entorno:

```bash
# .env.local
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=whsec_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_...

# Product IDs (crear en Stripe Dashboard)
STRIPE_PREMIUM_MONTHLY_PRICE_ID=price_...
STRIPE_PREMIUM_ANNUAL_PRICE_ID=price_...
STRIPE_ENTERPRISE_PRICE_ID=price_...
```

Crear productos en Stripe Dashboard:

```
Productos:
  1. DominioTotal Premium
     - Price: $40 USD / mes (recurring)
     - Price: $400 USD / año (recurring)

  2. DominioTotal Enterprise
     - Price: $150 USD / mes (recurring)
```

Paso 2: API Routes (2 horas)

app/api/stripe/create-checkout/route.ts

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import Stripe from 'stripe'
import { getAuthenticatedUser } from '@/lib/auth-server'
import { z } from 'zod'

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

const CheckoutSchema = z.object({
  priceId: z.string().startsWith('price_'),
  successUrl: z.string().url().optional(),
  cancelUrl: z.string().url().optional(),
})

export async function POST(req: NextRequest) {
  try {
    const user = await getAuthenticatedUser(req)
    if (!user) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }

    const body = await req.json()
    const { priceId, successUrl, cancelUrl } = CheckoutSchema.parse(body)

    // Verificar si ya tiene una suscripción activa
    const existingCustomer = await getStripeCustomerId(user.uid)

    const session = await stripe.checkout.sessions.create({
      customer: existingCustomer ?? undefined,
      customer_email: existingCustomer ? undefined : user.email!,
      mode: 'subscription',
      payment_method_types: ['card'],
      line_items: [{ price: priceId, quantity: 1 }],
      success_url: successUrl ?? `${process.env.NEXT_PUBLIC_APP_URL}/dashboard?upgrade=success`,
      cancel_url: cancelUrl ?? `${process.env.NEXT_PUBLIC_APP_URL}/pricing`,
      metadata: {
        userId: user.uid,
        firebaseUid: user.uid,
      },
      subscription_data: {
        metadata: {
          userId: user.uid,
        }
      },
      allow_promotion_codes: true,
      billing_address_collection: 'auto',
    })
```

```
    return NextResponse.json({ url: session.url })

  } catch (error) {
    if (error instanceof z.ZodError) {
      return NextResponse.json({ error: 'Invalid input' }, { status: 400 })
    }
    console.error('Stripe checkout error:', error)
    return NextResponse.json({ error: 'Internal error' }, { status: 500 })
  }
}

async function getStripeCustomerId(uid: string): Promise<string | null> {
  // Buscar en Firestore si ya tiene customerId
  const { adminDb } = await import('@/lib/firebase-admin')
  const userDoc = await adminDb.collection('users').doc(uid).get()
  return userDoc.data()?.stripeCustomerId ?? null
}
```

app/api/stripe/create-portal/route.ts

```typescript
```

```typescript
import { NextRequest, NextResponse } from 'next/server'
import Stripe from 'stripe'
import { getAuthenticatedUser } from '@/lib/auth-server'

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

export async function POST(req: NextRequest) {
  try {
    const user = await getAuthenticatedUser(req)
    if (!user) {
      return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
    }

    const { adminDb } = await import('@/lib/firebase-admin')
    const userDoc = await adminDb.collection('users').doc(user.uid).get()
    const stripeCustomerId = userDoc.data()?.stripeCustomerId

    if (!stripeCustomerId) {
      return NextResponse.json({ error: 'No subscription found' }, { status: 404 })
    }

    const session = await stripe.billingPortal.sessions.create({
      customer: stripeCustomerId,
      return_url: `${process.env.NEXT_PUBLIC_APP_URL}/dashboard`,
    })

    return NextResponse.json({ url: session.url })

  } catch (error) {
    console.error('Stripe portal error:', error)
    return NextResponse.json({ error: 'Internal error' }, { status: 500 })
  }
}
```

app/api/stripe/webhook/route.ts

```typescript
```

```
import { NextRequest, NextResponse } from 'next/server'
import Stripe from 'stripe'
import { adminDb } from '@/lib/firebase-admin'

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)

export async function POST(req: NextRequest) {
  const body = await req.text()
  const sig = req.headers.get('stripe-signature')!

  let event: Stripe.Event

  try {
    event = stripe.webhooks.constructEvent(
      body,
      sig,
      process.env.STRIPE_WEBHOOK_SECRET!
    )
  } catch (err) {
    console.error('Webhook signature failed:', err)
    return NextResponse.json({ error: 'Invalid signature' }, { status: 400 })
  }

  try {
    switch (event.type) {
      case 'checkout.session.completed': {
        const session = event.data.object as Stripe.CheckoutSession
        await handleCheckoutCompleted(session)
        break
      }

      case 'customer.subscription.updated': {
        const subscription = event.data.object as Stripe.Subscription
        await handleSubscriptionUpdated(subscription)
        break
      }

      case 'customer.subscription.deleted': {
        const subscription = event.data.object as Stripe.Subscription
        await handleSubscriptionCanceled(subscription)
        break
      }

      case 'invoice.payment_failed': {
        const invoice = event.data.object as Stripe.Invoice
        await handlePaymentFailed(invoice)
```

```
        break
      }
    }

    return NextResponse.json({ received: true })

  } catch (error) {
    console.error('Webhook handler error:', error)
    return NextResponse.json({ error: 'Handler failed' }, { status: 500 })
  }
}

async function handleCheckoutCompleted(session: Stripe.CheckoutSession) {
  const userId = session.metadata?.userId
  if (!userId || !session.customer || !session.subscription) return

  const subscription = await stripe.subscriptions.retrieve(
    session.subscription as string
  )
  const priceId = subscription.items.data[0].price.id
  const tier = getTierFromPriceId(priceId)

  await adminDb.collection('users').doc(userId).update({
    stripeCustomerId: session.customer as string,
    stripeSubscriptionId: session.subscription as string,
    subscription: {
      status: 'active',
      tier,
      priceId,
      currentPeriodEnd: new Date(subscription.current_period_end * 1000).toISOString(),
      cancelAtPeriodEnd: subscription.cancel_at_period_end,
    },
    updatedAt: new Date().toISOString(),
  })

  // Enviar email de bienvenida Premium
  await sendPremiumWelcomeEmail(userId, tier)
}

async function handleSubscriptionUpdated(subscription: Stripe.Subscription) {
  // Buscar usuario por stripeCustomerId
  const usersSnapshot = await adminDb
    .collection('users')
    .where('stripeCustomerId', '==', subscription.customer)
    .limit(1)
    .get()
```

```
  if (usersSnapshot.empty) return

  const userDoc = usersSnapshot.docs[0]
  const priceId = subscription.items.data[0].price.id
  const tier = getTierFromPriceId(priceId)

  await userDoc.ref.update({
    subscription: {
      status: subscription.status,
      tier,
      priceId,
      currentPeriodEnd: new Date(subscription.current_period_end * 1000).toISOString(),
      cancelAtPeriodEnd: subscription.cancel_at_period_end,
    },
    updatedAt: new Date().toISOString(),
  })
}

async function handleSubscriptionCanceled(subscription: Stripe.Subscription) {
  const usersSnapshot = await adminDb
    .collection('users')
    .where('stripeCustomerId', '==', subscription.customer)
    .limit(1)
    .get()

  if (usersSnapshot.empty) return

  await usersSnapshot.docs[0].ref.update({
    subscription: {
      status: 'canceled',
      tier: 'free',
      canceledAt: new Date().toISOString(),
    },
    updatedAt: new Date().toISOString(),
  })
}

async function handlePaymentFailed(invoice: Stripe.Invoice) {
  // Notificar al usuario que su pago falló
  // No cancelar inmediatamente (Stripe reintenta 3 veces)
  console.log(`Payment failed for customer: ${invoice.customer}`)
}

function getTierFromPriceId(priceId: string): 'premium' | 'enterprise' | 'free' {
  if (priceId === process.env.STRIPE_PREMIUM_MONTHLY_PRICE_ID) return 'premium'
  if (priceId === process.env.STRIPE_PREMIUM_ANNUAL_PRICE_ID) return 'premium'
  if (priceId === process.env.STRIPE_ENTERPRISE_PRICE_ID) return 'enterprise'
```

```
  return 'free'
}

async function sendPremiumWelcomeEmail(userId: string, tier: string) {
  // Usar el sistema de email existente (Resend)
  const { sendEmail } = await import('@/lib/mail')
  const userDoc = await adminDb.collection('users').doc(userId).get()
  const user = userDoc.data()

  if (!user?.email) return

  await sendEmail({
    to: user.email,
    subject: '🎉 Bienvenido a DominioTotal Premium',
    template: 'premium-welcome',
    data: { name: user.displayName, tier },
  })
}
```

Paso 3: Hook de Suscripción (1 hora)

hooks/useSubscription.ts

```typescript
```

```
import { useContext } from 'react'
import { AuthContext } from '@/contexts/AuthContext'

export type SubscriptionTier = 'free' | 'premium' | 'enterprise'

interface SubscriptionStatus {
  tier: SubscriptionTier
  isActive: boolean
  isPremium: boolean
  isEnterprise: boolean
  currentPeriodEnd?: string
  cancelAtPeriodEnd?: boolean
}

export function useSubscription(): SubscriptionStatus {
  const { user } = useContext(AuthContext)

  const subscription = user?.subscription
  const tier = subscription?.status === 'active' ? subscription.tier : 'free'

  return {
    tier,
    isActive: subscription?.status === 'active',
    isPremium: tier === 'premium' || tier === 'enterprise',
    isEnterprise: tier === 'enterprise',
    currentPeriodEnd: subscription?.currentPeriodEnd,
    cancelAtPeriodEnd: subscription?.cancelAtPeriodEnd,
  }
}

// Hook para actions de billing
export function useBilling() {
  const goToCheckout = async (priceId: string) => {
    const res = await fetch('/api/stripe/create-checkout', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ priceId }),
    })
    const { url } = await res.json()
    window.location.href = url
  }

  const goToPortal = async () => {
    const res = await fetch('/api/stripe/create-portal', {
      method: 'POST',
    })
```

```
    const { url } = await res.json()
    window.location.href = url
  }

  return { goToCheckout, goToPortal }
}
```

Paso 4: Feature Gates (1 hora)

components/PremiumGate.tsx

```typescript
```

```tsx
import { useSubscription } from '@/hooks/useSubscription'
import { Button } from '@/components/ui/button'
import { Lock } from 'lucide-react'

interface PremiumGateProps {
  children: React.ReactNode
  feature: string
  description?: string
}

export function PremiumGate({ children, feature, description }: PremiumGateProps) {
  const { isPremium } = useSubscription()

  if (isPremium) return <>{children}</>

  return (
    <div className="relative">
      <div className="blur-sm pointer-events-none select-none opacity-60">
        {children}
      </div>
      <div className="absolute inset-0 flex flex-col items-center justify-center bg-background/80 rounded-lg">
        <Lock className="w-8 h-8 text-muted-foreground mb-2" />
        <p className="font-semibold text-center">Feature Premium</p>
        {description && (
          <p className="text-sm text-muted-foreground text-center mt-1 px-4">
            {description}
          </p>
        )}
        <Button className="mt-4" size="sm" href="/pricing">
          Actualizar a Premium
        </Button>
      </div>
    </div>
  )
}
```

Uso en el dashboard:

```tsx
```

```
// Ocultar analytics avanzado para free users
<PremiumGate
  feature="advanced-analytics"
  description="Accedé a métricas de views, tasa de conversión y comparación con el mercado"
>
  <AdvancedAnalyticsDashboard />
</PremiumGate>
```

Paso 5: Página de Pricing (`app/pricing/page.tsx`)

```typescript
```

```
import { STRIPE_PRICES } from '@/lib/stripe-config'
import { PricingCard } from '@/components/pricing/PricingCard'
import { Check } from 'lucide-react'

export const metadata = {
  title: 'Planes y Precios | DominioTotal',
  description: 'Publicá propiedades gratis o upgrádate a Premium para destacarte.',
}

const plans = [
  {
    name: 'Free',
    price: 0,
    description: 'Para empezar sin riesgo',
    features: [
      '3 propiedades activas',
      '5 fotos por propiedad',
      'Leads ilimitados',
      'Dashboard básico',
      'Notificaciones email',
    ],
    cta: 'Empezar gratis',
    highlighted: false,
  },
  {
    name: 'Premium',
    price: 40,
    priceId: STRIPE_PRICES.PREMIUM_MONTHLY,
    annualPrice: 400,
    annualPriceId: STRIPE_PRICES.PREMIUM_ANNUAL,
    description: 'Para agentes activos que quieren más leads',
    features: [
      'Propiedades ilimitadas',
      '20 fotos por propiedad',
      'Analytics avanzado',
      'Badge Agente Verificado',
      '2 destacados/mes',
      'Prioridad en búsqueda',
      'CRM básico con pipeline',
      'Export de leads CSV',
      'Soporte prioritario',
    ],
    cta: 'Empezar Premium',
    highlighted: true,
    badge: 'MÁS POPULAR',
  },
```

```
]

export default function PricingPage() {
  return (
    <div className="container max-w-5xl py-16">
      <h1 className="text-4xl font-bold text-center mb-4">
        Planes que crecen con vos
      </h1>
      <p className="text-muted-foreground text-center mb-12 text-lg">
        Empezá gratis. Upgrádate cuando necesites más.
      </p>

      <div className="grid md:grid-cols-2 gap-8 max-w-3xl mx-auto">
        {plans.map((plan) => (
          <PricingCard key={plan.name} plan={plan} />
        ))}
      </div>

      {/* FAQ */}
      <div className="mt-16 text-center">
        <h2 className="text-2xl font-bold mb-8">Preguntas frecuentes</h2>
        {/* ... FAQ content */}
      </div>
    </div>
  )
}
```

Paso 6: Firestore Rules Actualizadas

```javascript
```

```
// firestore.rules — agregar validaciones de tier
function getUserTier(userId) {
  let userData = get(/databases/$(database)/documents/users/$(userId)).data;
  return userData.subscription != null &&
         userData.subscription.status == 'active'
         ? userData.subscription.tier
         : 'free';
}

function isPremiumOrHigher(userId) {
  return getUserTier(userId) in ['premium', 'enterprise'];
}

match /properties/{propId} {
  // Free: máximo 3 propiedades activas
  // Premium: ilimitadas
  allow create: if isAuthenticated() && isOwner(request.resource.data.userId) &&
    (isPremiumOrHigher(request.auth.uid) ||
     getUserActivePropertiesCount(request.auth.uid) < 3);
}
```

📊 Proyecciones de Revenue

Escenario Conservador (50% de agentes upgradean)

```
Mes 3:   5 agentes premium  × $40 = $200 MRR
Mes 4:   15 agentes premium × $40 = $600 MRR
Mes 5:   30 agentes premium × $40 = $1,200 MRR
Mes 6:   50 agentes premium × $40 = $2,000 MRR
Mes 9:  150 agentes premium × $40 = $6,000 MRR
Mes 12: 300 agentes premium × $40 = $12,000 MRR
        +50 enterprise      × $150 = $7,500 MRR
        TOTAL                      = $19,500 MRR
```

Break-even

```
Costos infrastructure Mes 6: ~$300/mes
Revenue Mes 6:               ~$2,000/mes
Profit Mes 6:                ~$1,700/mes (85% margin)
```

📅 Roadmap de Implementación

Semana 1: Backend

- [ ] Crear productos en Stripe Dashboard
- [ ] Implementar `/api/stripe/create-checkout`
- [ ] Implementar `/api/stripe/create-portal`
- [ ] Implementar `/api/stripe/webhook`
- [ ] Configurar webhook en Stripe Dashboard
- [ ] Actualizar Firestore con campos de suscripción

Semana 2: Frontend + Polish

- [ ] Crear `useSubscription` hook
- [ ] Crear `PremiumGate` component
- [ ] Crear página `/pricing`
- [ ] Agregar banners de upgrade en dashboard
- [ ] Actualizar Firestore Rules
- [ ] Testing manual del flujo completo
- [ ] Email de bienvenida Premium

Semana 3: Go-Live

- [ ] Testing en Stripe Test Mode
- [ ] Switch a Stripe Live Mode
- [ ] Anunciar a beta testers (email)
- [ ] Monitorear primeras conversiones

🧪 Testing del Flujo de Pago

```bash
# Tarjetas de test de Stripe
✅ Pago exitoso:    4242 4242 4242 4242
❌ Pago declinado:  4000 0000 0000 0002
⚠️ Auth requerida:  4000 0025 0000 3155
```

Checklist de Testing Manual

- [ ] Hacer click en "Upgrade" → redirige a Stripe Checkout
- [ ] Completar pago con tarjeta de test → redirección a /dashboard?upgrade=success
- [ ] Dashboard muestra badge Premium inmediatamente
- [ ] Features bloqueadas ahora están disponibles
- [ ] Portal de billing funciona (actualizar tarjeta, cancelar)
- [ ] Webhook actualiza Firestore correctamente

- [ ] Al cancelar → tier vuelve a Free al final del período

⚠️ Consideraciones Legales (Uruguay)

1. **IVA:** Stripe puede manejar colección de impuestos automáticamente para el mercado internacional. Para Uruguay se recomienda consultar con un contador.
2. **Términos de Servicio:** Actualizar TOS con sección de suscripciones y refunds.
3. **Política de Cancelación:** Implementar "cancel at period end" (no cancelación inmediata) para buena experiencia de usuario.
4. **Facturación:** Stripe genera facturas automáticamente por email.

Documento creado: 18 Febrero 2026
Owner: Founder / Producto
Próxima revisión: Mes 3 (pre-launch Premium)